

public boolean equals(Object obj) :

Predefined non static method of Object class.

* It is used to compare two **custom objects** based on the **memory reference OR based on the address** because internally equals(Object obj) method of Object class is using == operator only as shown in the program below

```
package com.ravi.equals_demo;

class Player
{
    private int id;
    private String name;

    public Player(int id, String name)
    {
        super();
        this.id = id;
        this.name = name;
    }
}

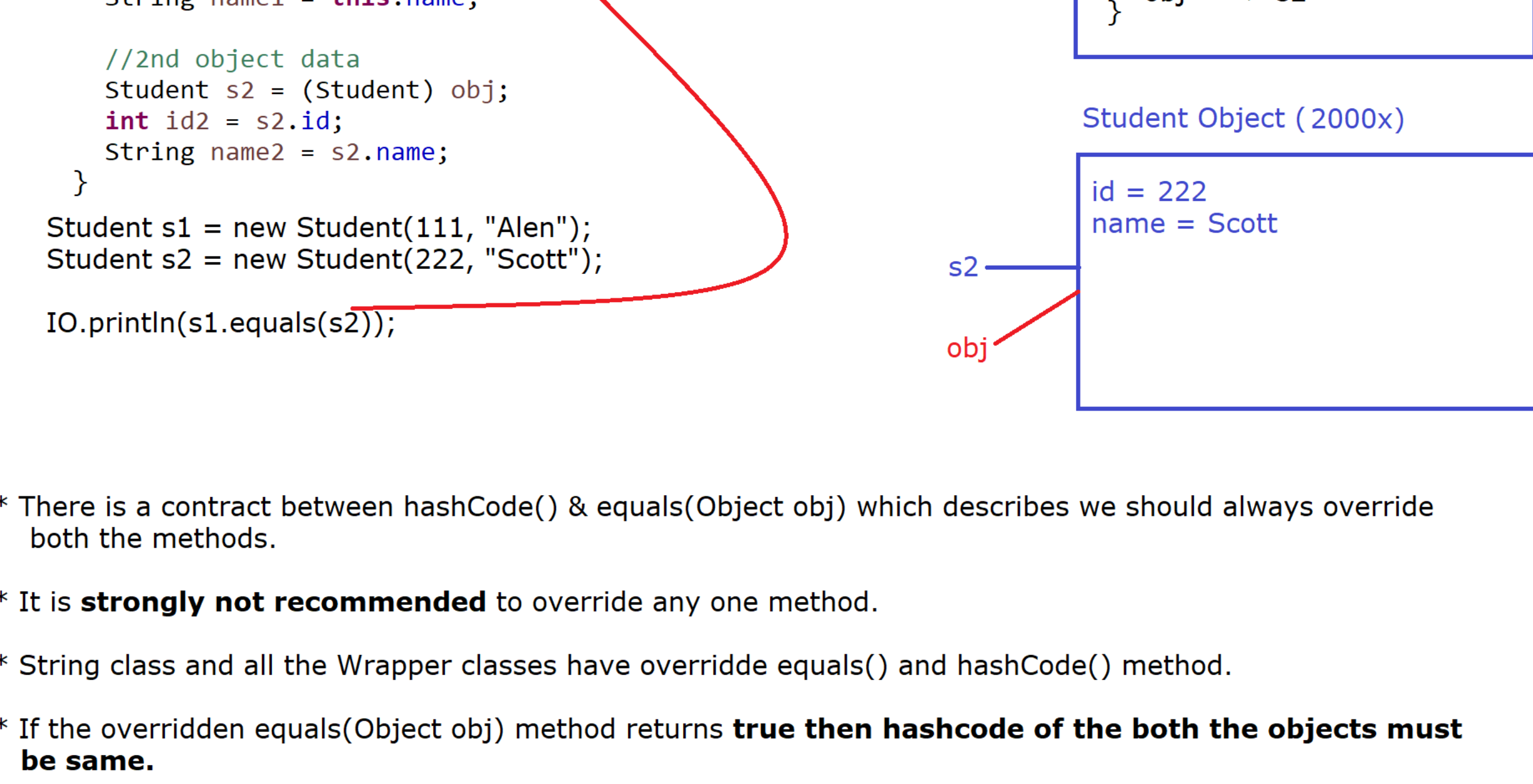
public class EqualsDemo1 {

    public static void main(String[] args)
    {
        Player p1 = new Player(111, "Scott");
        Player p2 = new Player(111, "Scott");

        IO.println(p1==p2); //false
        IO.println(p1.equals(p2)); //false
    }
}
```

* If we want to compare two custom objects based on the **content** but not based on the memory reference OR address then **we should override Object class equals(Object obj) for content comparison as shown in the diagram below.**

Diagram for understanding equals() method Logic :



* There is a contract between hashCode() & equals(Object obj) which describes we should always override both the methods.

* It is **strongly not recommended** to override any one method.

* String class and all the Wrapper classes have override equals() and hashCode() method.

* If the overridden equals(Object obj) method returns **true** then **hashcode of the both the objects must be same.**

```
package com.ravi.equals_demo;

class Student
{
    private int id;
    private String name;

    public Student(int id, String name)
    {
        super();
        this.id = id;
        this.name = name;
    }

    @Override
    public int hashCode()
    {
        return this.id + this.name.hashCode();
    }

    //Override equals(Object obj) method for content comparison
    @Override
    public boolean equals(Object obj)    //obj = s2
    {
        //Retrieve 1st and 2nd object data
        //1st object data
        int id1 = this.id;
        String name1 = this.name;

        //2nd object data
        Student s2 = (Student) obj;
        int id2 = s2.id;
        String name2 = s2.name;

        if(id1==id2 && name1.equals(name2))
        {
            return true;
        }
        else
        {
            return false;
        }
    }
}

public class EqualsDemo2
{
    public static void main(String[] args)
    {
        Student s1 = new Student(111, "Alen");
        Student s2 = new Student(111, "Alen");

        IO.println(s1.equals(s2)); //true

        IO.println(s1.hashCode());
        IO.println(s2.hashCode());
    }
}
```

```
package com.ravi.equals_demo;

class Product
{
    private int id;
    private String name;

    public Product(int id, String name)
    {
        super();
        this.id = id;
        this.name = name;
    }

    @Override
    public int hashCode()
    {
        return this.id + this.name.hashCode();
    }

    @Override
    public boolean equals(Object obj)
    {
        Product p2 = (Product) obj;

        if(this.id==p2.id && this.name.equals(p2.name))
        {
            return true;
        }
        else
        {
            return false;
        }
    }
}

public class EqualsDemo3
{
    public static void main(String[] args)
    {
        Product p1 = new Product(111, "Laptop");
        Product p2 = new Product(111, "Laptop");

        IO.println(p1.equals(p2));
        IO.println(p1.hashCode());
        IO.println(p2.hashCode());
    }
}
```

Note : null with instanceof operator always returns false.

```
package com.ravi.equals_demo;

class Employee
{
    private int id;
    private String name;

    public Employee(int id, String name)
    {
        super();
        this.id = id;
        this.name = name;
    }

    @Override
    public int hashCode()
    {
        return this.id + this.name.hashCode();
    }

    @Override
    public boolean equals(Object obj)
    {
        if(obj instanceof Employee e2)
        {
            if(this.id ==e2.id && this.name.equals(e2.name))
            {
                return true;
            }
            else
            {
                return false;
            }
        }
        else
        {
            System.err.println("Comparison is not possible");
            return false;
        }
    }
}

public class EqualsDemo4 {

    public static void main(String[] args)
    {
        Employee e1 = new Employee(111, "Scott");
        Manager m1 = new Manager(111, "Scott");

        IO.println(e1.equals(m1));
        IO.println(e1.equals(null));
    }
}

class Manager
{
    private int id;
    private String name;

    public Manager(int id, String name)
    {
        super();
        this.id = id;
        this.name = name;
    }
}
```